

## Lecture Note: Git & GitHub – Basics



Nishut Suman

31 Dec, 2025 At 6:00 PM

Notes

Foundation

Recommended

Resource



Associated Lectures (1)



Description



Discussions

# Lecture Notes: Git & GitHub Basics

**Prerequisites:** Understanding of what a file, folder, and command line are.  
Basic knowledge of Python or any programming language (helpful but not mandatory).

### What you'll be able to do:

- Explain what Git and GitHub are and how they differ.
- Apply Git commands to initialize, track, and commit code.
- Create and manage branches for parallel development.
- Use **.gitignore** correctly to keep repositories clean.
- Identify and correct common beginner mistakes in version control.

## 1. Introduction - What is Git & GitHub and Why Should You Care?

### Core Definition

**Git** is a distributed version control system that records every change you make to your files, allowing you to restore, compare, and collaborate.

**GitHub** is an online hosting platform that stores Git repositories in the cloud and enables teamwork through collaboration features like pull requests and issue tracking.

### A Simple Analogy

Think of **Git** as a *personal time machine* for your projects-every commit is a snapshot in time.

**GitHub** is like *Google Drive for code*-it stores, syncs, and lets others collaborate.



### Limitation:

Unlike Drive, Git tracks **how** and **why** changes happened-it's not just storage.

### Why This Matters to You

**Problem it solves:** Losing progress, overwriting files, or not knowing what changed.

**What you'll gain:**

- Confidence in experimenting without fear of breaking things.
- Team-ready workflows for professional environments.
- Transparent history for debugging and learning.

**Real-world context:**

Every serious software project-Linux, Android, VS Code-is powered by **Git**.

---

## 2. The Foundation – Core Concepts Explained

---

### Concept A: Version Control System (VCS)

**Definition:** A tool that keeps track of changes to files over time so you can revert to earlier versions, collaborate, and manage code history.

**Key characteristics:**

- Tracks every change line-by-line.
- Works locally without needing the internet.
- Enables collaboration through branching & merging.

**Example:** Adding a new feature, then rolling back when a bug appears.

**Common confusion:** Many think “**saving**” equals version control. It's not. Git tracks *each change's context*-who, when, and why.

---

### Concept B: Git and GitHub

**Definition:**

- **Git:** The local tool installed on your computer.
- **GitHub:** The online platform for storing and sharing Git repositories.

**Relation:** You use Git commands locally → then connect (push/pull) to GitHub for collaboration.

**Key characteristics:**

- Git works offline; GitHub adds cloud sync.
- Commits live locally until pushed.
- GitHub enables pull requests, issues, and remote access.

**Example:** You create a repo using **git init**, then upload it to GitHub using **git remote add origin**.

**Remember:** Git ≠ GitHub. One is a **tool**, the other a **service** built around it.

---

## Concept C: The Three-Stage Workflow

**Definition:** The heart of Git's workflow:

- 1 **Working Directory** – Where files are edited.
- 2 **Staging Area** – Where changes wait for review (git add).
- 3 **Repository** – Where commits are permanently recorded (git commit).

**Example:**

```
git add hello.py
git commit -m "Added greeting script"
```

**Common confusion:**

Beginners assume **git add** saves changes - it doesn't. Only **git commit** does.

---

## Concept D: Branching & Merging

**Definition:** Branching lets you create an independent line of development to work on features without affecting the main branch.

Merging brings those changes back safely.

**Key commands:**

```
git branch new-feature
git checkout new-feature
git merge new-feature
```

**Analogy:** Think of branches as *parallel universes*-you can experiment freely. Merging is when the universes reunite.

**Common confusion:** Learners often modify the **main** branch directly. Always create a branch first to isolate work.

---

## Concept E: .gitignore File

**Definition:** A text file listing which files/folders Git should skip (e.g., temporary files, passwords, build artifacts).

**Example:**

```
__pycache__/
.env
*.log
```

**Key point:**

**.gitignore** prevents sensitive or irrelevant files from polluting your repo.

---

## How They Work Together

Git creates and stores **commits** locally, organizes them via **branches**, filters unwanted data using **.gitignore**, and syncs with the cloud via **GitHub** - forming a complete version-control ecosystem.

---

## 3. Seeing It in Action - Worked Examples

---

### Example 1: Your First Repository

**Scenario:** You want to start tracking your Python project.

**Steps:**

```
mkdir student-project
cd student-project
git init
echo "print('Hello, Git!')" > app.py
git add .
git commit -m "Initial commit"
```

**Output:**

```
[master (root-commit) 1a2b3c4] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 app.py
```

**What just happened:** Git created a new repository, staged your file, and recorded the first snapshot.

**Check your understanding:**

What is the purpose of **git add** vs **git commit**?

---

### Example 2: Branching for a New Feature

**Scenario:** You're adding a login feature without disturbing main.

```
git branch login-feature
git checkout login-feature
# modify app.py
git add app.py
git commit -m "Added login feature"
git checkout main
```

```
git merge login-feature
```

**Output:**

```
Updating 1a2b3c4..5d6e7f8
Fast-forward
 app.py | 5 +++++
 1 file changed, 5 insertions(+)
```

**Key lesson:** Branches enable safe, isolated experimentation-merging integrates them cleanly.

### Example 3: Ignoring Unwanted Files

**Scenario:** You don't want `__pycache__` or `.env` tracked.

```
echo "__pycache__/" >> .gitignore
echo ".env" >> .gitignore
git add .gitignore
git commit -m "Added .gitignore file"
```

**Result:** Git now skips those paths permanently.

## 4. Common Pitfalls and How to Avoid Them

- **Mistake 1:**

Forgetting to configure Git ([user.name](#) and `user.email`)

- **Why it's a problem:** Commits appear as anonymous or cause sync issues on GitHub.
- **Fix:**

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

- **Why it works:** Every commit now carries an identifiable author.

- **Mistake 2:**

Running `git add .` without checking what changed

- **Why it's a problem:** Unintended or secret files get committed.

- **Right approach:** Run **git status** first, then selectively add files:

```
git add app.py
```

- **Mistake 3:**

Skipping commit messages or writing vague ones

- **Problem:** History becomes meaningless.
- **Fix:** Use descriptive messages like "Refactored user validation logic"
- **Why:** Good commit messages make teamwork and debugging faster.

- **Mistake 4:**

Editing directly on the main branch

- **Problem:** Breaks production code.
- **Fix:** Create a feature branch first.
- **Command:** `git checkout -b feature-name`

- **Mistake 5:**

Forgetting to create a **.gitignore**

- **Problem:** Sensitive files get pushed online.
- **Fix:** Always add **.gitignore** before your first commit.

---

## 5. Practice & Self-Assessment

---

### Practice Task

**Challenge:** Create a repo for your Student OOP project.

- Initialize Git.
- Add and commit your Python files.
- Add a **.gitignore** to exclude **\*\*\*pycache\*\*\*** and **.env**
- Make a second commit after a small code change.
- Check history with **git log**

**Hint:** Use **git status** often to see what changed before committing.

**Extension:** Push it to GitHub and share the URL with a peer.

---

### Check Your Understanding

**Explain:** Why is version control superior to saving multiple file copies like **final\_v2.py**?

**Apply:** If you accidentally deleted a function, how could Git help recover it?

**Error analysis:** What's wrong with this sequence?

```
git add .  
git init  
git commit -m "Initial commit"
```

**Transfer:** How would you use branching if two people were working on different features simultaneously?

### Answers & Explanations:

Git records incremental history and metadata for each change-manual versions don't.

You can restore a previous commit with **git checkout** or **git revert**

Initialization must come first; you can't add or commit before **git init**

Each creates a branch; merge when finished to combine work.

---

## 6. Consolidation - Key Takeaways & Next Steps

---

### The Essential Ideas

- **Git = local version control; GitHub = remote collaboration.**
- **Commits** are your project's checkpoints-always write clear messages.
- **Branching** lets you experiment without risk.
- **.gitignore** keeps repos clean and secure.

### Mental Model Check

---

Think of Git as a movie timeline-you can always rewind to any scene.

GitHub is like a **cinema** where everyone can watch, comment, and edit together.

### What You Can Now Do

---

You can initialize repos, commit meaningful changes, create branches, and safely share code on GitHub. These are skills used daily by every professional developer.

### Next Steps

- Learn about **remote repositories** (**git push**, **git pull**, **git clone**).
- Explore **merge conflicts** and how to resolve them.
- Study **branching strategies** (feature branch, Git Flow, etc.).

### Resources:

- [Pro Git Book - Chapters 2-3](#)
- [Learn Git Branching \(Interactive Game\)](#) (\*\*Must Try)

---

## Installation Video Guide:

---

Watch these quick installation videos for your OS:

- [For Windows](#)
- [For macOS](#)

---

## Quick Reference Card

---

| Action          | Command             | Purpose                   |
|-----------------|---------------------|---------------------------|
| Initialize repo | git init            | Start version control     |
| Stage file      | git add filename    | Move file to staging area |
| Commit          | git commit -m "msg" | Record permanent snapshot |
| Create branch   | git branch name     | New development line      |
| Switch branch   | git checkout name   | Move to another branch    |
| Merge           | git merge name      | Combine branch changes    |
| Log             | git log             | View commit history       |
| Ignore files    | .gitignore          | Exclude unwanted files    |

---

### If you're stuck:

Run **git status** - it tells you exactly where you are.

---

✅ You've now crossed from "learning Git" to **thinking like a developer** who controls history rather than being controlled by it. 🚀